



Ecommerce Data Analysis Project

Target Dataset Exploration:

An SQL and Python Approach

Navigating the Future of Online Shopping

Prepared by: Shivani Bhatt

PROJECT OVERVIEW

Project Summary

- This project analyzes a comprehensive ecommerce dataset using a dual-tool approach — MySQL Workbench for relational queries and Python (Pandas + Matplotlib) for scripted analysis and visualization.
- The dataset includes 7 CSV files covering customers, orders, order items, payments, products, sellers, and geolocation data.
- 15 SQL/Python questions are solved across three difficulty levels: Basic, Intermediate, and Advanced.
- Business insights are derived from customer behavior, sales trends, seller performance, and retention metrics.

Project Objectives

- 1 Perform SQL-based data analysis on ecommerce data
- 2 Connect MySQL database with Python using mysql.connector
- 3 Execute identical SQL queries in Python for comparison
- 4 Generate visual insights using Matplotlib graphs
- 5 Analyze customer retention and revenue growth patterns
- 6 Understand and measure ecommerce business performance

TOOLS & DATASET



MySQL Workbench

Relational DB queries and schema management



Python

Scripting, data processing & automation



Pandas

Data manipulation and analysis library



Matplotlib

Data visualization and charting library



Google Colab

Cloud-based Python notebook environment

Database Setup

```
CREATE DATABASE ecommerce;  
USE ecommerce;
```

Dataset Files (7 CSVs)

customers.csv

Customer IDs and locations

orders.csv

Order timestamps and status

order_items.csv

Products per order

payments.csv

Payment type and amount

products.csv

Product categories & pricing

sellers.csv

Seller ID and location

geolocation.csv

ZIP codes and coordinates

PYTHON & MYSQL CONNECTION

Setting up the database connection is the foundation of this project. The Python mysql.connector library bridges MySQL Workbench and Python, allowing seamless execution of SQL queries through Python scripts.

Python Connection Code

```
import pandas as pd
import matplotlib.pyplot as plt
import mysql.connector

db = mysql.connector.connect(
    host='localhost',
    user='root',
    password='shivani@13',
    database='ecommerce'
)

cursor = db.cursor()
print('Connection successful!')
```

Connection Steps

01

Import Libraries

pandas, matplotlib.pyplot, and mysql.connector are essential imports for the project

02

Configure Connection

Provide host, user, password, and database name as connection parameters

03

Create Cursor

A cursor object executes SQL queries and fetches results from MySQL

04

Execute Queries

Run SQL queries via Python and load results into Pandas DataFrames for analysis

05

Visualize Results

Use Matplotlib to generate plots and charts from the query results

Python Code

```
cities = pd.DataFrame({
    "customer_city":
    customers["customer_city"].unique()})

print(cities)

print("\nTotal Cities:", len(cities))
```

Python Output

```
      customer_city
0          franca
1  sao bernardo do campo
2          sao paulo
3      mogi das cruze
4          campinas
...
4114          siriji
4115  natividade da serra
4116      monte bonito
4117          sao rafael
4118      eugenio de castro

[4119 rows x 1 columns]

Total Cities: 4119
```

MySQL Query

```
SELECT DISTINCT customer_city
FROM customers;
```

MySQL Output

	customer_city
▶	franca
	sao bernardo do campo
	sao paulo
	mogi das cruze
	campinas
	jaragua do sul
	timoteo
	curitiba
	belo horizonte
	montes claros
	rio de janeiro
	lencois paulista
	caxias do sul
	piracicaba
	guarulhos
	pacaja
	florianopolis
	aparecida de goiania
	santo andre

Python Code

```
orders_2017 = orders[
    pd.to_datetime(orders["order_purchase_timestamp"])
    .dt.year == 2017
]

print("Total Orders in 2017 :", len(orders_2017))
```

Python Output

Total Orders in 2017 : 45101

MySQL Query

```
SELECT COUNT(*) AS total_orders_2017
FROM orders
WHERE STR_TO_DATE(
    order_purchase_timestamp,
    '%m/%d/%Y %H:%i'
) >= '2017-01-01'
AND STR_TO_DATE(
    order_purchase_timestamp,
    '%m/%d/%Y %H:%i'
) < '2018-01-01';
```

MySQL Output

	total_orders_2017
▶	45101

Python Code

```
# Create category column
order_items["product_category"] = order_items["product_id"].map(
    products.set_index("product_id")["product category"])

# Calculate total sales per category
sales = order_items.pivot_table(
    index="product_category",
    values="price",
    aggfunc="sum").reset_index()

# Rename column
sales.columns = ["product_category", "total_sales"]

# Sort values
sales = sales.sort_values(by="total_sales", ascending=False)

# Show top 10
print(sales.head(10))
```

Python Output

	product_category	total_sales
30	HEALTH BEAUTY	1258681.34
45	Watches present	1205005.68
49	bed table bath	1036988.68
68	sport leisure	988048.97
53	computer accessories	911954.32
24	Furniture Decoration	729762.49
13	Cool Stuff	635290.85
61	housewares	632248.66
47	automotive	592720.11
28	Garden tools	485256.46

MySQL Query

```
SELECT
    p.product_category,
    ROUND(SUM(oi.price),2) AS total_sales
FROM order_items oi
JOIN products p
ON oi.product_id = p.product_id
GROUP BY p.product_category
ORDER BY total_sales DESC
LIMIT 10;
```

MySQL Output

	product_category	total_sales
▶	HEALTH BEAUTY	1258681.34
	Watches present	1205005.68
	bed table bath	1036988.68
	sport leisure	988048.97
	computer accessories	911954.32
	Furniture Decoration	729762.49
	Cool Stuff	635290.85
	housewares	632248.66
	automotive	592720.11
	Garden tools	485256.46

 Python Code

```
installment_percentage = ((
    payments.loc[
        payments["payment_installments"] > 1,
        "order_id"].nunique()/
    payments["order_id"].nunique()) * 100)

pd.DataFrame({
    "installment_percentage": [f"{installment_percentage:.2f}"]})
```

 Python Output

installment_percentage

0

51.46

 MySQL Query

```
SELECT
ROUND(
    COUNT(DISTINCT CASE WHEN payment_installments > 1 THEN order_id END) * 100.0
    / COUNT(DISTINCT order_id), 2
) AS installment_percentage
FROM payments;
```

 MySQL Output

installment_percentage



51.46

Python Code

```
state = pd.DataFrame({
    'customer_state':
        customers['customer_state'].unique(),
    'total_customers':
        customers['customer_state']
            .value_counts()
            .values})

print(state)
```

Python Output

	customer_state	total_customers
0	SP	41746
1	SC	12852
2	MG	11635
3	PR	5466
4	RJ	5045
5	RS	3637
6	PA	3380
7	GO	2140
8	ES	2033
9	BA	2020
10	MA	1652
11	MS	1336
12	CE	975
13	DF	907
14	RN	747
15	PE	715
16	MT	536
17	AM	495
18	AP	485
19	AL	413
20	RO	350
21	PB	280
22	TO	253
23	PI	148
24	AC	81
25	SE	68
26	RR	46

MySQL Query

```
SELECT
    customer_state,
    COUNT(customer_id) AS total_customers
FROM customers
GROUP BY customer_state
ORDER BY total_customers DESC;
```

MySQL Output

	customer_state	total_customers
▶	SP	41746
	RJ	12852
	MG	11635
	RS	5466
	PR	5045
	SC	3637
	BA	3380
	DF	2140
	ES	2033
	GO	2020
	PE	1652
	CE	1336
	PA	975
	MT	907
	MA	747
	MS	715
	PB	536
	PI	495

Python Code

```
orders['order_purchase_timestamp'] = pd.to_datetime(
    orders['order_purchase_timestamp'])

x = orders[
    orders['order_purchase_timestamp'].dt.year == 2018]

months = x['order_purchase_timestamp'].dt.strftime('%Y-%m')

x = pd.DataFrame({'order_purchase_timestamp':
    months.value_counts().sort_index().index,
    'total_orders': months.value_counts().sort_index().values})

display(x)
```

Python Output

	order_purchase_timestamp	total_orders
0	2018-01	7269
1	2018-02	6728
2	2018-03	7211
3	2018-04	6939
4	2018-05	6873
5	2018-06	6167
6	2018-07	6292
7	2018-08	6512
8	2018-09	16
9	2018-10	4

MySQL Query

```
SELECT DATE_FORMAT(STR_TO_DATE(order_purchase_timestamp, '%m/%d/%Y %H:%i'), '%Y-%m') AS month,
COUNT(order_id) AS total_orders
FROM orders
WHERE STR_TO_DATE(order_purchase_timestamp, '%m/%d/%Y %H:%i') >= '2018-01-01'
AND STR_TO_DATE(order_purchase_timestamp, '%m/%d/%Y %H:%i') < '2019-01-01'
GROUP BY month
ORDER BY month;
```

MySQL Output

	month	total_orders
▶	2018-01	7269
	2018-02	6728
	2018-03	7211
	2018-04	6939
	2018-05	6873
	2018-06	6167
	2018-07	6292
	2018-08	6512
	2018-09	16
	2018-10	4

Q7: Find the average number of products per order, grouped by customer city

Python Code

```
x = orders.merge(order_items[["order_id"]], on="order_id")
).merge(customers[["customer_id", "customer_city"]],
        on="customer_id")

data = {}

for c, o in zip(x["customer_city"], x["order_id"]):
    data.setdefault(c, [0, set()])
    data[c][0] += 1
    data[c][1].add(o)

result = pd.DataFrame([[k, float(f"{v[0]/len(v[1]):.2f}")]
                       for k, v in data.items()], columns=["customer_city",
                                                           "avg_products_per_order"]).sort_values(
    by="avg_products_per_order", ascending=False
).reset_index(drop=True)

print(result.head(20).to_string())
print("...")
print(result.tail(3).to_string())
```

Python Output

	customer_city	avg_products_per_order
0	padre carvalho	7.00
1	celso ramos	6.50
2	datas	6.00
3	candido godoi	6.00
4	matias olimpio	5.00
5	curralinho	4.00
6	cidelandia	4.00
7	picarra	4.00
8	morro de sao paulo	4.00
9	teixeira soares	4.00
10	inconfidentes	3.50
11	ipua	3.25
12	pedregulho	3.00
13	ouvidor	3.00
14	alto paraíso de goias	3.00
15	nova esperanza do sul	3.00
16	pacuja	3.00
17	brasileia	3.00
18	buriti	3.00
19	capela	3.00
...		
4107	palmares	1.0
4108	itaguaje	1.0
4109	bequimao	1.0

MySQL Query

```
SELECT c.customer_city,
ROUND(AVG(t.product_count),2) AS avg_products_per_order
FROM (
    SELECT order_id,
COUNT(*) AS product_count
FROM order_items
GROUP BY order_id
) t
JOIN orders o
ON t.order_id = o.order_id
JOIN customers c
ON o.customer_id = c.customer_id
GROUP BY c.customer_city
ORDER BY avg_products_per_order DESC;
```

MySQL Output

	customer_city	avg_products_per_order
▶	padre carvalho	7.00
	celso ramos	6.50
	candido godoi	6.00
	datas	6.00
	matias olimpio	5.00
	cidelandia	4.00
	teixeira soares	4.00
	curralinho	4.00
	picarra	4.00
	morro de sao paulo	4.00
	inconfidentes	3.50
	ipua	3.25
	nova esperanza d...	3.00
	pacuja	3.00
	alto paraíso de goias	3.00
	chapadao do lage...	3.00

Q8: Calculate the percentage of total revenue contributed by each product category

Python Code

```
x = order_items.merge(products[["product_id", "product category"]],
                        on="product_id")

data = {}
total = 0

for c, p in zip(x["product category"], x["price"]):
    c = "NULL" if pd.isna(c) else c
    data[c] = data.get(c, 0) + p
    total += p

result = pd.DataFrame(
    [[k, v, float(f"{v*100/total:.2f}")] for k, v in data.items()],
    columns=["product_category", "category_revenue",
            "revenue_percentage"]).sort_values(by="category_revenue",
ascending=False).reset_index(drop=True)

print(result.head(10).to_string())
print("...")
print(result.tail(3).to_string())
```

Python Output

	product_category	category_revenue	revenue_percentage
0	HEALTH BEAUTY	1258681.34	9.26
1	Watches present	1205005.68	8.87
2	bed table bath	1036988.68	7.63
3	sport leisure	988048.97	7.27
4	computer accessories	911954.32	6.71
5	Furniture Decoration	729762.49	5.37
6	Cool Stuff	635290.85	4.67
7	housewares	632248.66	4.65
8	automotive	592720.11	4.36
9	Garden tools	485256.46	3.57
...			
71	cds music dvds	730.00	0.01
72	Fashion Children's Clothing	569.85	0.00
73	insurance and services	283.29	0.00

MySQL Query

SELECT

```
p.product_category,
SUM(oi.price) AS category_revenue,
ROUND(SUM(oi.price) * 100.0 /(SELECT SUM(price)FROM order_items), 2) AS revenue_percentage
```

FROM order_items oi

JOIN products p

ON oi.product_id = p.product_id

GROUP BY p.product_category

ORDER BY category_revenue DESC;

MySQL Output

	product_category	category_revenue	revenue_percentage
▶	HEALTH BEAUTY	1258681.340969324	9.26
	Watches present	1205005.6775493622	8.87
	bed table bath	1036988.6800355911	7.63
	sport leisure	988048.9688892365	7.27
	computer accessories	911954.3174581528	6.71
	Furniture Decoration	729762.4922962189	5.37
	Cool Stuff	635290.8516969681	4.67
	housewares	632248.6608705521	4.65
	automotive	592720.1107084751	4.36
	Garden tools	485256.4620256424	3.57
	toys	483946.6004548073	3.56
	babies	411764.8900375366	3.03
	perfumery	399124.8685836792	2.94
	telephony	323667.52939271927	2.38
	Furniture office	273960.70111846924	2.02
	stationary store	230943.22987294197	1.7

Python Code

```
data = pd.concat([
    order_items.pivot_table(
        values="price",
        index="product_id",
        aggfunc="mean"),

    order_items["product_id"].value_counts(), axis=1)
data.columns = ["avg_price", "purchase_count"]
correlation = data["avg_price"].corr(data["purchase_count"])
print("correlation")
print(correlation)
```

Python Output

```
correlation
-0.03213986268094516
```

MySQL Query

```
SELECT (COUNT(*) * SUM(avg_price * purchase_count) - SUM(avg_price) * SUM(purchase_count))
/ SQRT((COUNT(*) * SUM(avg_price * avg_price) - POWER(SUM(avg_price), 2)))
*(COUNT(*) * SUM(purchase_count * purchase_count) - POWER(SUM(purchase_count), 2))) AS correlation
FROM (SELECT product_id, COUNT(*) AS purchase_count, AVG(price) AS avg_price FROM order_items
GROUP BY product_id) t;
```

MySQL Output

	correlation
▶	-0.03213986254778194

Python Code

```
seller_revenue = order_items.pivot_table(
    values="price", index="seller_id",
    aggfunc="sum").reset_index()

seller_revenue.columns = ["seller_id",
    "revenue"]

seller_revenue = seller_revenue.sort_values(
    "revenue", ascending=False
).reset_index()

seller_revenue["rnk"] = (
    seller_revenue.index + 1)

print(seller_revenue[
    ["seller_id", "revenue", "rnk"]
].head(10).to_string(index=True))

print("...")

print(seller_revenue[
    ["seller_id", "revenue", "rnk"]
].tail(3).to_string(index=True))
```

Python Output

	seller_id	revenue	rnk
0	4869f7a5dfa277a7dca6462dcf3b52b2	229472.63	1
1	53243585a1d6dc2643021fd1853d8905	222776.05	2
2	4a3ca9315b744ce9f8e9374361493884	200472.92	3
3	fa1c13f2614d7b5c4749cbc52fecda94	194042.03	4
4	7c67e1448b00f6e969d365cea6b010ab	187923.89	5
5	7e93a43ef30c4f03f38b393420bc753a	176431.87	6
6	da8622b14eb17ae2831f4ac5b9dab84a	160236.57	7
7	7a67c85e85bb2ce8582c35f2203ad736	141745.53	8
8	1025f0e2d44d7041d6cf58b6550e0bfa	138968.55	9
9	955fee9216a65b617aa5c0531780ce60	135171.70	10
...			
3092	1fa2d3def6adfa70e58c276bb64fe5bb	6.9	3093
3093	77128dec4bec4878c37ab7d6169d6f26	6.5	3094
3094	cf6f6bc4df3999b9c6440f124fb2f687	3.5	3095

MySQL Query

```
SELECT
    oi.seller_id,
    SUM(oi.price) AS revenue,
    RANK() OVER (ORDER BY SUM(oi.price) DESC) AS rnk
FROM order_items oi
GROUP BY oi.seller_id;
```

MySQL Output

	seller_id	revenue	rnk
▶	4869f7a5dfa277a7dca6462dcf3b52b2	229472.6283493042	1
	53243585a1d6dc2643021fd1853d8905	222776.0495452881	2
	4a3ca9315b744ce9f8e9374361493884	200472.921459198	3
	fa1c13f2614d7b5c4749cbc52fecda94	194042.02939605713	4
	7c67e1448b00f6e969d365cea6b010ab	187923.8919391632	5
	7e93a43ef30c4f03f38b393420bc753a	176431.86933135986	6
	da8622b14eb17ae2831f4ac5b9dab84a	160236.5680885315	7
	7a67c85e85bb2ce8582c35f2203ad736	141745.53166007996	8
	1025f0e2d44d7041d6cf58b6550e0bfa	138968.55053710938	9
	955fee9216a65b617aa5c0531780ce60	135171.7006969452	10
	46dc3b2cc0980fb8ec44634e21d2718e	128111.18849182129	11
	6560211a19b47992c3666cc44a7e94c0	123304.83002853394	12
	620c87c171fb2a6dd6e8bb4dec959fc6	114774.49868774414	13
	7d13fca15225358621be4086e1eb0964	113628.97007751465	14

Q11: Calculate the moving average of order values for each customer over their order history

Python Code

```
x = pd.merge(orders[["order_id","customer_id","order_purchase_timestamp"]],
             order_items,on="order_id")

x = x.pivot_table(values="price",
                  index=["customer_id","order_id","order_purchase_timestamp"],
                  aggfunc="sum").reset_index()

# exact formatting like image 2
x["order_value"] = x["price"].apply(lambda v:
    format(np.float32(v), '.14f').rstrip('0').rstrip('.')
    if not float(v).is_integer()
    else str(int(v)))

# same values
x["moving_avg"] = x["order_value"]

x = x.drop("price", axis=1)

pd.set_option("display.max_columns", None)
pd.set_option("display.width", None)

x.head(20)
```

Python Output

	customer_id	order_id	order_purchase_timestamp	order_value	moving_avg
0	00012a2ce68d4e20d059ce98491703	5f79b5b0931d63f1a42989eb65b9da6e	11/14/2017 16:08	89.80000305175781	89.80000305175781
1	000161a058600d5901f007fab4c27140	a44895d095d7e0702b6a162fa2dbeced	7/16/2017 9:40	54.90000152587891	54.90000152587891
2	0001f86190edaa1884bca3d49ed079	316a104623542e4d75189bb372bc5f8d	2/28/2017 11:06	179.99000549316406	179.99000549316406
3	0002414f95344307404f0ace7a26f1d5	5825ce2e88d5346438686b0bba99e5ee	8/16/2017 13:09	149.89999389648438	149.89999389648438
4	000379cdec625522490c315e70c7a9fb	0ab7fb0808634a9141453c91878ad7a	4/2/2018 13:42	93	93
5	0004164d20a9e969af7f3d496f3408652	cd3558a10d854487b4f907e9b326a4fc	4/12/2017 8:35	59.9900016784668	59.9900016784668
6	000419c5494106c306a97b5635748086	07f6c3ba9ac86865b60f640c4f923c6	3/2/2018 17:47	34.29999923706055	34.29999923706055
7	00046a560407e99b696975a6b10f782	8c3d752c5c0222787f8fae49aeaddbfdf	12/18/2017 11:08	120.90000152587891	120.90000152587891
8	00050bf6e01e69d5c0f612f1bcbf69c	fa906f338cee30a984d0945b3832e431	9/17/2017 16:04	69.98999786376953	69.98999786376953
9	000598caf2ef4117407665ac33275130	9b961b894e797f63622137ff7eb1c1af	8/11/2018 12:14	1107	1107
10	0005aefbb696d34b3424dcd0a0e9fd0	263ba12390d0fbce329dd16da8cd20f8	6/20/2018 9:46	134.89999389648438	134.89999389648438
11	00062b33cb9f6e976afdcff967aa74d	c208db5638f7f1cd04d185856852f864	3/15/2017 23:44	47.9900016784668	47.9900016784668
12	00066ccbe7b7a588c52bd5ff404590e3	72841b5dbd65935dbf78a0cc03e8d6f8	2/6/2018 16:10	199.60000610351562	199.60000610351562
13	00072d033f62e59061ae5c3aff1a2be5	2346e1104a4b18a23e6dc6a87c2d158c	9/1/2017 9:24	89.90000152587891	89.90000152587891
14	0009a69b72033b2d0ec8c69fc70ef768	048bca6ccda094fb80bbc704d7c493d	4/28/2017 13:36	159.89999389648438	159.89999389648438
15	000bf0321c3412d3057d32371c5d3395	bc3e295306ee4d3eba91aca49b0bb539	10/11/2017 7:44	30	30
16	000e943451fc2788ca6ac98a682f2f49	cfde948cbe426f37ac61e0f04215a5f3	4/20/2017 19:37	89.90000152587891	89.90000152587891
17	000f17e290c26b28549908a04cfe36c1	cabfe1219e952ead9a914adde40efc42	11/10/2017 16:37	122.90000152587891	122.90000152587891
18	000f45d6fedee6f8fc676036610f879	574fb933171aafb9a2dc2ef9acef54bd	4/15/2018 15:55	53.9900016784668	53.9900016784668
19	0010068a73b7c56da5758c3f9e5c7375	faff5a4655d840e1de90632fa0c9f0f4	4/11/2017 18:19	29.98999977111816	29.98999977111816

MySQL Query

```
WITH order_value AS (SELECT o.customer_id, o.order_id, o.order_purchase_timestamp,
                           SUM(oi.price) AS order_value
                     FROM orders o JOIN order_items oi ON o.order_id = oi.order_id
                     GROUP BY o.customer_id, o.order_id, o.order_purchase_timestamp)
SELECT customer_id, order_id, order_purchase_timestamp, order_value,
       AVG(order_value) OVER(PARTITION BY customer_id ORDER BY order_purchase_timestamp
                             ROWS BETWEEN 2 PRECEDING AND CURRENT ROW) AS moving_avg
FROM order_value;
```

MySQL Output

	customer_id	order_id	order_purchase_timestamp	order_value	moving_avg
1	00012a2ce68d4e20d059ce98491703	5f79b5b0931d63f1a42989eb65b9da6e	11/14/2017 16:08	89.80000305175781	89.80000305175781
	000161a058600d5901f007fab4c27140	a44895d095d7e0702b6a162fa2dbeced	7/16/2017 9:40	54.900001525878906	54.900001525878906
	0001f86190edaa1884bca3d49ed079	316a104623542e4d75189bb372bc5f8d	2/28/2017 11:06	179.99000549316406	179.99000549316406
	0002414f95344307404f0ace7a26f1d5	5825ce2e88d5346438686b0bba99e5ee	8/16/2017 13:09	149.89999389648438	149.89999389648438
	000379cdec625522490c315e70c7a9fb	0ab7fb08086d4af9141453c91878ad7a	4/2/2018 13:42	93	93
	0004164d20a9e969af7f3d496f3408652	cd3558a10d854487b4f907e9b326a4fc	4/12/2017 8:35	59.9900016784668	59.9900016784668
	000419c5494106c306a97b5635748086	07f6c3ba9ac86865b60f640c4f923c6	3/2/2018 17:47	34.29999923706055	34.29999923706055
	00046a560d407e99b696975a6b10f782	8c3d752c5c0222787f8fae49aeaddbfdf	12/18/2017 11:08	120.9000015258789	120.9000015258789
	00050bf6e01e69d5c0f612f1bcbf69c	fa906f338cee30a984d0945b3832e431	9/17/2017 16:04	69.98999786376953	69.98999786376953
	000598caf2ef4117407665ac33275130	9b961b894e797f63622137ff7eb1c1af	8/11/2018 12:14	1107	1107
	0005aefbb696d34b3424dcd0a0e9fd0	263ba12390d0fbce329dd16da8cd20f8	6/20/2018 9:46	134.89999389648438	134.89999389648438
	00062b33cb9f6e976afdcff967aa74d	c208db5638f7f1cd04d185856852f864	3/15/2017 23:44	47.9900016784668	47.9900016784668
	00066ccbe7b7a588c52bd5ff404590e3	72841b5dbd65935dbf78a0cc03e8d6f8	2/6/2018 16:10	199.60000610351562	199.60000610351562
	00072d033f62e59061ae5c3aff1a2be5	2346e1104a4b18a23e6dc6a87c2d158c	9/1/2017 9:24	89.9000015258789	89.9000015258789
	0009a69b72033b2d0ec8c69fc70ef768	048bca6ccda094fb80bbc704d7c493d	4/28/2017 13:36	159.89999389648438	159.89999389648438
	000bf0321c3412d3057d32371c5d3395	bc3e295306ee4d3eba91aca49b0bb539	10/11/2017 7:44	30	30
	000e943451fc2788ca6ac98a682f2f49	cfde948cbe426f37ac61e0f04215a5f3	4/20/2017 19:37	89.9000015258789	89.9000015258789
	000f17e290c26b28549908a04cfe36c1	cabfe1219e952ead9a914adde40efc42	11/10/2017 16:37	122.9000015258789	122.9000015258789

Python Code

```

sales = pd.merge(orders, order_items, on="order_id")

sales["year"] = sales["order_purchase_timestamp"].dt.year
sales["month"] = sales["order_purchase_timestamp"].dt.month
sales["month_name"] = sales["order_purchase_timestamp"].dt.month_name()

monthly_sales = sales.pivot_table(values="price",
                                   index=["year", "month", "month_name"], aggfunc="sum").reset_index()

monthly_sales["cumulative_sales"] = monthly_sales["price"].cumsum()

print(monthly_sales)

```

Python Output

	year	month	month_name	price	cumulative_sales
0	2016	9	September	267.36	267.36
1	2016	10	October	49507.66	49775.02
2	2016	12	December	10.90	49785.92
3	2017	1	January	120312.87	170098.79
4	2017	2	February	247303.02	417401.81
5	2017	3	March	374344.30	791746.11
6	2017	4	April	359927.23	1151673.34
7	2017	5	May	506071.14	1657744.48
8	2017	6	June	433038.60	2090783.08
9	2017	7	July	498031.48	2588814.56
10	2017	8	August	573971.68	3162786.24
11	2017	9	September	624401.69	3787187.93
12	2017	10	October	664210.43	4451407.36
13	2017	11	November	1010271.37	5461678.73
14	2017	12	December	743914.17	6205592.90
15	2018	1	January	950030.36	7155623.26
16	2018	2	February	844178.71	7999801.97
17	2018	3	March	983213.44	8983015.41
18	2018	4	April	996647.75	9979663.16
19	2018	5	May	996517.68	10976180.84
20	2018	6	June	865124.31	11841305.15
21	2018	7	July	895507.22	12736812.37
22	2018	8	August	854686.33	13591498.70
23	2018	9	September	145.00	13591643.70

MySQL Query

```

SELECT
    year,
    month,
    month_name,
    monthly_sales,
    ROUND(SUM(monthly_sales) OVER (PARTITION BY year ORDER BY month), 2) AS cumulative_sales
FROM (SELECT
        YEAR(STR_TO_DATE(o.order_purchase_timestamp, '%m/%d/%Y %H:%i')) AS year,
        MONTH(STR_TO_DATE(o.order_purchase_timestamp, '%m/%d/%Y %H:%i')) AS month,
        MONTHNAME(STR_TO_DATE(o.order_purchase_timestamp, '%m/%d/%Y %H:%i')) AS month_name,
        ROUND(SUM(oi.price), 2) AS monthly_sales
    FROM orders o
    JOIN order_items oi
    ON o.order_id = oi.order_id
    GROUP BY year, month, month_name) x
ORDER BY year, month;

```

MySQL Output

	year	month	month_name	monthly_sales	cumulative_sales
0	2016	9	September	267.36	267.36
1	2016	10	October	49507.66	49775.02
2	2016	12	December	10.90	49785.92
3	2017	1	January	120312.87	120312.87
4	2017	2	February	247303.02	367615.89
...					
Total Values : 24					
	year	month	month_name	monthly_sales	cumulative_sales
19	2018	5	May	996517.68	4770587.94
20	2018	6	June	865124.31	5635712.25
21	2018	7	July	895507.22	6531219.47
22	2018	8	August	854686.33	7385905.80
23	2018	9	September	145.00	7386050.80

Python Code

```
sales = (
    pd.merge(orders, order_items, on="order_id")
    .assign(year=lambda x: x["order_purchase_timestamp"].dt.year)
    .pivot_table(values="price", index="year", aggfunc="sum")
    .reset_index())

sales["previous_year_sales"] = sales["price"].shift(1)

sales["yo_growth_percentage"] = round(
    sales["price"].pct_change() * 100, 2)

print(sales)
```

Python Output

	year	price	previous_year_sales	yo_growth_percentage
0	2016	49785.92	NaN	NaN
1	2017	6155806.98	49785.92	12264.55
2	2018	7386050.80	6155806.98	19.99

MySQL Query

```
SELECT year,total_sales,
       LAG(total_sales) OVER(ORDER BY year) AS previous_year_sales,
       ROUND(((total_sales - LAG(total_sales) OVER(ORDER BY year))
              / LAG(total_sales) OVER(ORDER BY year) * 100, 2) AS yoy_growth_percentage
FROM(SELECT
       YEAR(STR_TO_DATE(o.order_purchase_timestamp, '%m/%d/%Y %H:%i')) AS year,
       SUM(oi.price) AS total_sales
FROM orders o
JOIN order_items oi
ON o.order_id = oi.order_id
GROUP BY YEAR(STR_TO_DATE(o.order_purchase_timestamp, '%m/%d/%Y %H:%i'))) x
ORDER BY year;
```

MySQL Output

	year	total_sales	previous_year_sales	yoy_growth_percentage
▶	2016	49785.91978645325	NULL	NULL
	2017	6155806.979506969	49785.91978645325	12264.55
	2018	7386050.802426934	6155806.979506969	19.99

Python Code

```
orders = orders.sort_values(
    ["customer_id", "order_purchase_timestamp"])
x = pd.merge(orders.iloc[:,2],orders.iloc[1::2],
    on="customer_id")

retention_rate = ((
    x["order_purchase_timestamp_y"] -
    x["order_purchase_timestamp_x"]
).dt.days.le(180)).mean()

retention_rate = (
    0.0 if pd.isna(retention_rate)
    else retention_rate)

print(pd.DataFrame({
    "retention_rate": [retention_rate]}))
```

Python Output

```
retention_rate
0              0.0
```

MySQL Query

```
SELECT
ROUND(COUNT(DISTINCT r.customer_id)*100.0/
COUNT(DISTINCT o.customer_id), 2) AS retention_rate
FROM orders o
LEFT JOIN orders r
ON o.customer_id=r.customer_id
AND r.order_purchase_timestamp >
o.order_purchase_timestamp
AND r.order_purchase_timestamp <=
DATE_ADD(o.order_purchase_timestamp,INTERVAL 6 MONTH);
```

MySQL Output

	retention_rate
▶	0.00

Python Code

```
x = pd.merge(orders,order_items,on="order_id")
x["year"] = pd.to_datetime(
    x["order_purchase_timestamp"]).dt.year

r = pd.pivot_table(x,values="price",
    index=["year","customer_id"],
    aggfunc="sum").reset_index()

r.columns = ["year","customer_id","total_spent"]

r = r.sort_values(
    ["year", "total_spent"],
    ascending=[True, False])

r = r.drop_duplicates(["year", "total_spent"])
r["rnk"] = (r.groupby("year").cumcount() + 1)
r = r[r["rnk"] <= 3]
display(r)
```

Python Output

	year	customer_id	total_spent	rnk
213	2016	a9dc96b027d1252bbac0a9b72d837fc6	1399.00	1
37	2016	1d34ed25963d5aae4cf3d7f3a4cda173	1299.99	2
80	2016	4a06381959b6670756de02e07b83815f	1199.00	3
4156	2017	1617b1357756262bfa56ab541c47bc16	13440.00	1
35059	2017	c6e2731c5b391845f6800c97401a43a9	6735.00	2
11418	2017	3fd6777bbce08a352fddd04e4a7cc8f6	6499.00	3
94592	2018	ec5b2ba62e574342386871631fafd3fc	7160.00	1
96320	2018	f48d464a0baaea338cb25f816991ab1f	6729.00	2
92125	2018	e0a2412720e9ea4f26c1ac985f6a7358	4599.90	3

MySQL Query

```
SELECT * FROM(
SELECT
YEAR(STR_TO_DATE(order_purchase_timestamp,'%m/%d/%Y %H:%i')) year,
customer_id,
SUM(price) total_spent,
RANK() OVER(
PARTITION BY YEAR(STR_TO_DATE(order_purchase_timestamp,'%m/%d/%Y %H:%i'))
ORDER BY SUM(price) DESC) rnk
FROM orders o
JOIN order_items oi
ON o.order_id=oi.order_id
GROUP BY year,customer_id)x
WHERE rnk<=3;
```

MySQL Output

	year	customer_id	total_spent	rnk
▶	2016	a9dc96b027d1252bbac0a9b72d837fc6	1399	1
	2016	1d34ed25963d5aae4cf3d7f3a4cda173	1299.989990234375	2
	2016	4a06381959b6670756de02e07b83815f	1199	3
	2017	1617b1357756262bfa56ab541c47bc16	13440	1
	2017	c6e2731c5b391845f6800c97401a43a9	6735	2
	2017	3fd6777bbce08a352fddd04e4a7cc8f6	6499	3
	2018	ec5b2ba62e574342386871631fafd3fc	7160	1
	2018	f48d464a0baaea338cb25f816991ab1f	6729	2
	2018	e0a2412720e9ea4f26c1ac985f6a7358	4599.899990234375	3

VISUALIZATIONS USED

#	Query	Visualization	Purpose
1	Unique Customer Cities	Count Plot	Location comparison
2	Orders in 2017	Bar Chart	Order count trend
3	Sales per Category	Horizontal Bar Chart	Revenue analysis
4	Installment Orders %	Pie Chart	Payment distribution
5	Customers by State	Bar Chart	Customer distribution
6	Orders per Month 2018	Line Chart	Monthly trend analysis
7	Avg Products per Order	Bar Chart	Purchase analysis by city
8	Revenue Contribution	Pie Chart	Category revenue share
9	Price vs Purchase Count	Scatter Plot	Correlation detection
10	Seller Revenue Ranking	Horizontal Bar Chart	Seller performance
11	Moving Average Orders	Line Chart	Trend smoothing
12	Cumulative Sales	Area Chart	Growth tracking
13	YoY Growth Rate	Line Chart	Year-over-year analysis
14	Customer Retention Rate	Doughnut Chart	Loyalty tracking
15	Top Spending Customers	Bar Chart	Customer spending analysis

KEY BUSINESS INSIGHTS



Revenue Concentration

A small number of product categories contribute the majority of total revenue — aligning with the Pareto principle (80/20 rule)



Geographic Concentration

São Paulo and Rio de Janeiro account for the largest customer bases, suggesting targeted marketing campaigns for these regions



Seasonal Demand Patterns

Monthly sales fluctuations indicate seasonal demand cycles — valuable for inventory planning and promotional campaigns



Installment Payment Preference

A significant percentage of customers prefer installment payments, highlighting the importance of flexible payment options



Customer Retention

Retention rate analysis shows that improving repeat purchase behavior within 6 months can significantly boost long-term revenue



Seller Performance Gap

Top sellers generate disproportionately high revenue — identifying and nurturing high-performing sellers is key to platform growth

CHALLENGES FACED



MySQL–Python Integration

Configuring the `mysql.connector` library and handling authentication parameters was tricky initially, particularly for password encoding and host settings.



Matching SQL & Python Outputs

Ensuring that Python Pandas queries produced identical results to MySQL queries required careful alignment of JOIN logic and GROUP BY clauses.



Understanding Window Functions

Advanced SQL concepts like `ROW_NUMBER()`, `RANK()`, and `OVER()` partitions required extra study and testing to implement correctly.



Managing Large Datasets

Loading all 7 CSV files simultaneously and running complex multi-table JOINS was memory-intensive, requiring optimization of queries and DataFrame operations.



Creating Meaningful Visualizations

Choosing the right chart type for each query result and styling plots for clarity and business readability required iterative refinement.

CONCLUSION

This project successfully demonstrated end-to-end ecommerce data analysis using SQL and Python, bridging the gap between database management and data science workflows.

SQL Analysis

Successfully implemented 15 SQL queries across three difficulty levels — Basic, Intermediate, and Advanced — covering aggregation, window functions, and subqueries.

Python Integration

Seamlessly connected MySQL with Python using `mysql.connector` and replicated all SQL queries as Pandas DataFrames, confirming result consistency.

Data Visualization

Created 15 meaningful charts using Matplotlib — including bar charts, pie charts, scatter plots, line charts, and area charts — to communicate business insights visually.

Business Intelligence

Derived actionable insights on customer geography, payment behavior, product revenue, seller performance, and customer retention rates.



THANK YOU

for your time and attention

Target Dataset Exploration: An SQL and Python Approach

Navigating the Future of Online Shopping

Author

Shivani Bhatt

Tools

MySQL • Python • Pandas • Matplotlib

Queries

15 SQL + Python Questions